

주차장 출입 차량 번호판 인식 시스템

김찬종 2026.04 – 2026.06

사용기술 및 환경

분야	사용 기술
Programming	Python
Object Detection	YOLO26s (Training), YOLO26x (Vehicle Detection)
Computer Vision	OpenCV(Perspective Transform, Coordinate Extraction)
OCR	PaddleOCR
Dataset	Roboflow
Training Environment	Google Colab

인식 대상 번호판

대한민국 차량 번호판
(1996년대 부터~현재)을
대상으로 했습니다.

추출된 번호판	번호	추출된 번호판	번호
---------	----	---------	----



91다4471



71주9578



43우2075



21모0162



73사3018



367다2418



32사1751



135나4015



34나5380

차 번호판 훈련 데이터 셋

roboflow

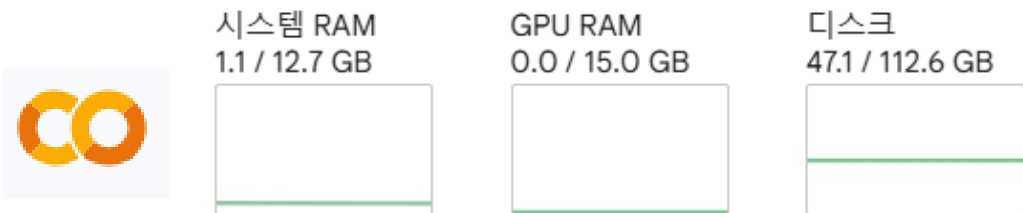
Train- 1050 개

Valid - 309 개

Test - 152 개

♣ Train, Valid, Test 데이터 셋은 roboflow 사이트에서 가져왔습니다.

훈련 실행 및 환경



런타임 유형 변경

런타임 유형

Python 3

하드웨어 가속기 ?



CPU



T4 GPU

훈련 실행 코드

```
from ultralytics import YOLO
```

```
# 모델 불러오기 (사전학습 모델)
```

```
model = YOLO("yolo26s.pt")
```

```
# 8. 학습
```

```
model.train(
```

```
    data="/content/drive/MyDrive/YOLO_backup/license_plate_v1/yolo26/data.yaml",
```

```
    epochs=100,
```

```
    imgsz=960,
```

```
    batch=8,
```

```
    degrees=5,
```

```
    translate=0.5,
```

```
    scale=0.9,
```

```
    shear=10,
```

```
    perspective=0.0001,
```

```
    mosaic=1,
```

```
)
```

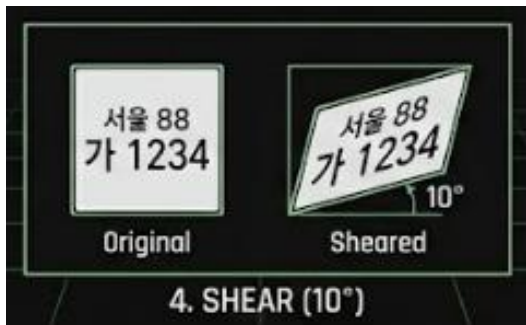
데이터 증강(Augmentation) 설정

degrees=5: 이미지를 ± 5 도 정도 까지 살짝 회전시킵니다. (기울어진 번호판 대비)

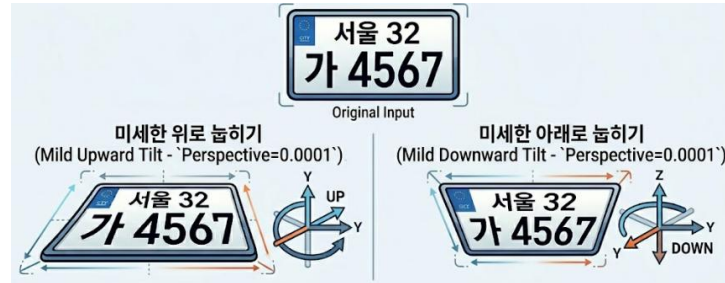
translate=0.5: 이미지를 상하좌우로 50%까지 이동시킵니다. (번호판 위치가 바뀔 때 대비)

scale=0.9: 이미지를 0.9배 이내의 비율로 크기를 조절합니다. (멀리 있는 번호판 대비)

shear=10: 컴퓨터에게 정면 사진만 주면 옆에서 찍은 번호판을 못 알아보기 때문에, 이미지를 좌우 또는 상하로 10도만큼 밀어서 찌그러뜨린(평행사변형 모양) 상태로 공부를 시키는 것입니다.



perspective=0.0001: 이미지를 아주 미세하게 위/아래로 눕히는 효과입니다.



주차장 차단기 카메라가 차량을 내려다볼 때 생기는 원근 왜곡을 AI가 스스로 극복하게 만들어서, 실제 현장에서 어떤 각도로 찍히든 번호판을 잘 찾아내도록 하기 위함.

mosaic=1: 4장의 이미지를 한 장으로 합쳐서 학습하는 기법입니다. 복잡한 배경에서 모델이 객체(번호판)를 잘 찾게 하기 위함.



Validation 데이터(309개) 검증 및 성능 분석

구분	실제 번호판 (True: LP)	실제 배경 (True: background)
번호판으로 예측	309	2
(Predicted: LP)	(True Positive / 정답)	(False Positive / 오탐지)
배경으로 예측	0	0
(Predicted: background)	(False Negative / 미검출 제로)	(True Negative)

✔ Validation Performance Metrics

검증 데이터 309장을 이용하여 번호판 검출 모델의 성능을 평가하였다.

🕒 Accuracy

99.36%

전체 예측 정확도

🔍 Recall

100.00%

미검출(False Negative) 0건

🎯 Precision

99.36%

오탐(False Positive) 최소화

📊 F1-Score

99.68%

Precision-Recall 균형 지표

Test 데이터(152개) 검증 및 성능 분석

구분	실제 번호판 (True: LP)	실제 배경 (True: background)
번호판으로 예측	152	1
(Predicted: LP)	(True Positive / 정답)	(False Positive / 오탐지)
배경으로 예측	0	0
(Predicted: background)	(False Negative / 미검출 제로)	(True Negative)

학습에 사용되지 않은 테스트 데이터 152장으로 일반화 성능을 평가하였다.

📌 Accuracy

99.35%

전체 예측 정확도

🎯 Precision

99.34%

오탐(False Positive) 1건

🔍 Recall

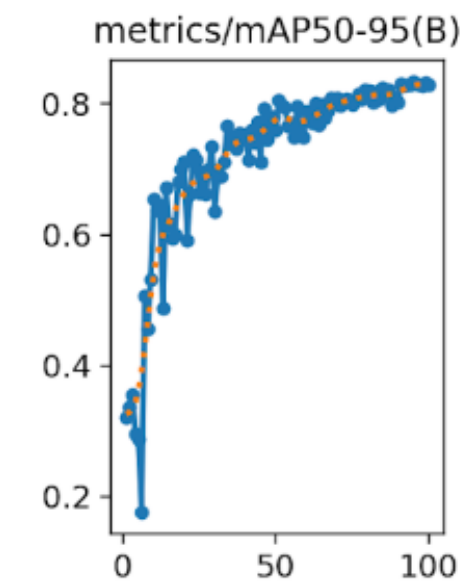
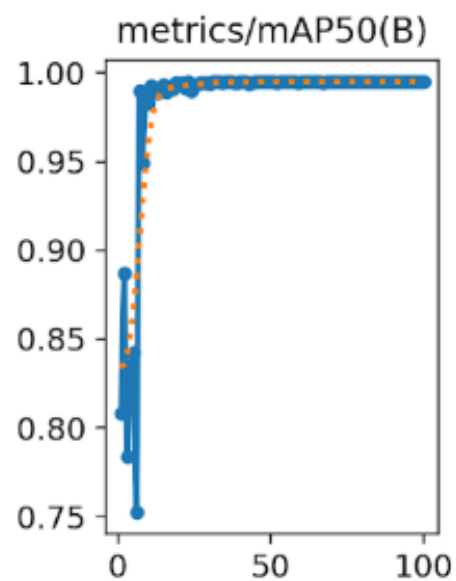
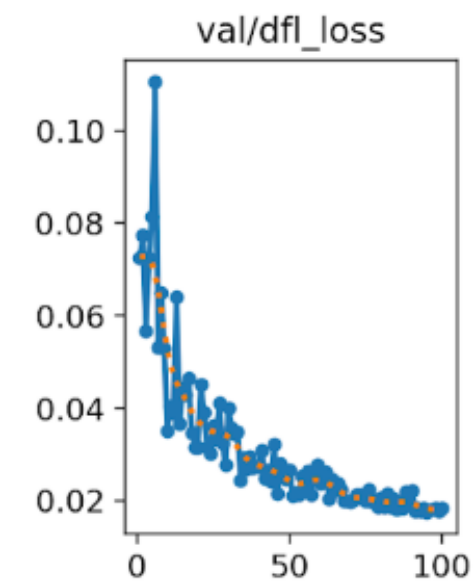
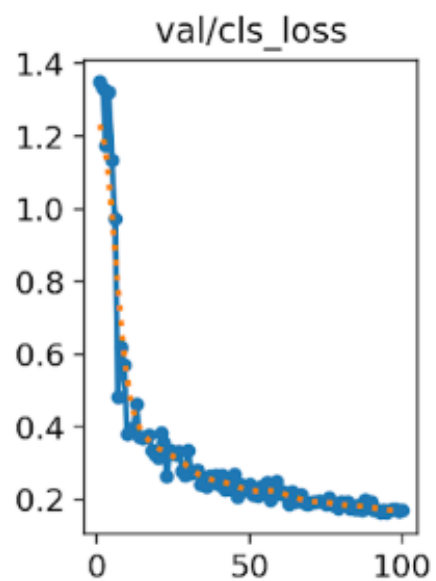
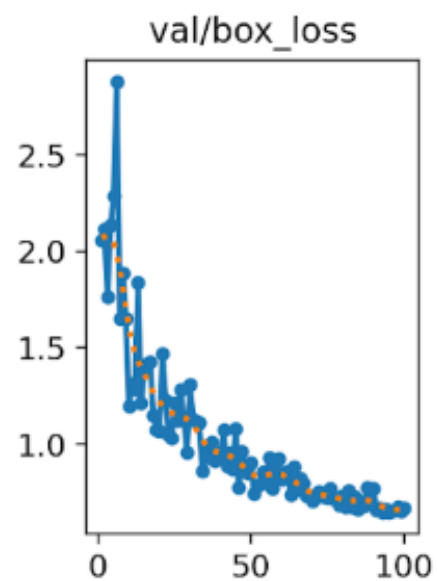
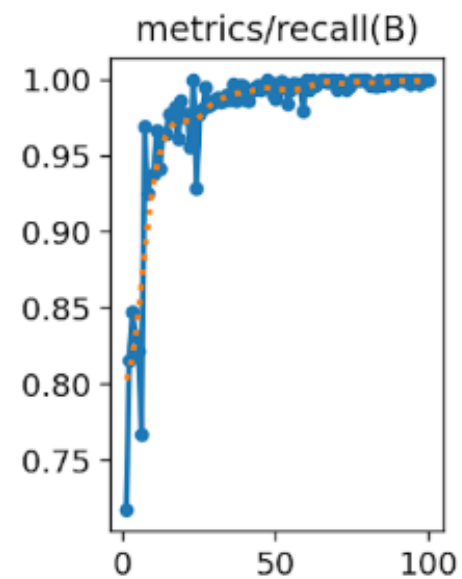
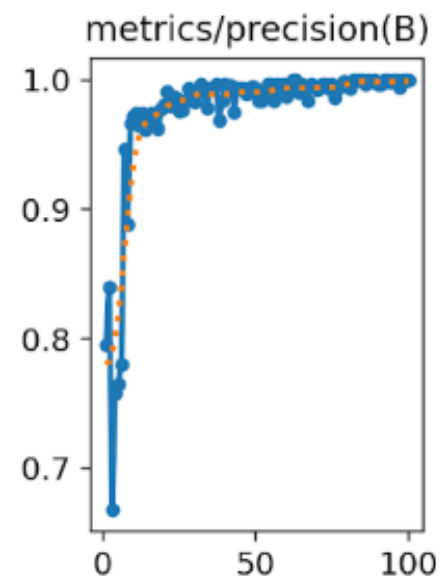
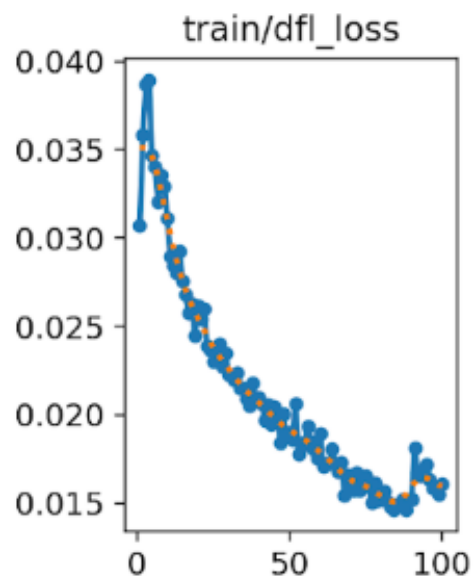
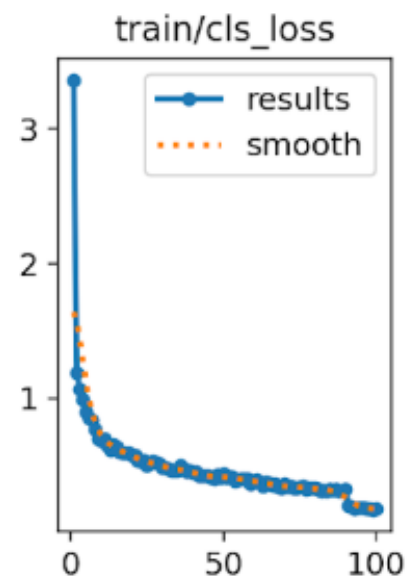
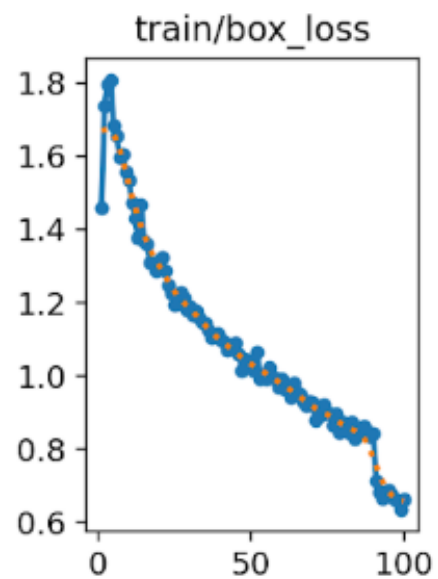
100.00%

미검출(False Negative) 0건

📊 F1-Score

99.67%

균형 성능 유지



번호판 검출 모델(best.pt)만 입출 차 사진을 적용한 결과

Plate Detection



Plate Detection



Plate Detection



주차장 출입 자동차 번호판 탐지
(yolo26x 모델 + (best.pt) 모델)

Original Image



주차장 출입 차량을 촬영한 원본 이미지 입력 단계입니다.

Vehicle Detection



객체 탐지에 특화된 **yolo26x** 모델을 활용하여 이미지 내에서 **차량 (car, bus, truck)** 객체를 우선 탐색하고 바운딩 박스(Green Box)를 생성합니다.
카메라와 가장 가까운 차량을 식별하기 위해 **면적 (Area)**이 가장 큰 객체를 타겟팅 하도록 설계했습니다.

in/out Vehicle Crop



Crop된 이미지에 번호판 검출 전용 모델 (**best.pt**)을 적용합니다.

Plate Detection In in/out Vehicle



검출이 된 부분 중 가장 확률이 높은 부분을 crop

Final Plate



검출된 차량 번호판

자동차 번호판 번호 읽기

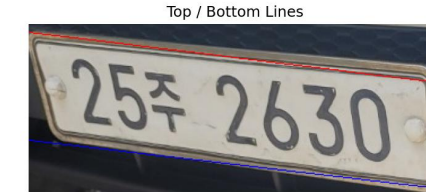
원근 변환을 위한
진짜 번호판 영역의 가로 직선(**Top Line**), 세로 직선(**Bottom Line**) 추출



검출된 번호판에 1차
paddle ocr을 적용

Page 14~Page 15 참조

Top Line, **Bottom Line** 추출



검출된 번호판에 1차
paddle ocr을 적용

Page 14~Page 15 참조

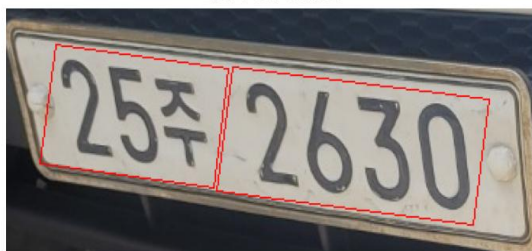
Top Line, **Bottom Line** 추출

이미지에 담긴 핵심 과정은 PaddleOCR로 인식한 글자 상자(Box)들을 기준 삼아,
번호판의 가장 위쪽(**Top Line**)과 아래쪽(**Bottom Line**) 경계를 찾는 방법입니다.

1st OCR Boxes



1st OCR Boxes



1. 기본 개념 (영역 좌표)

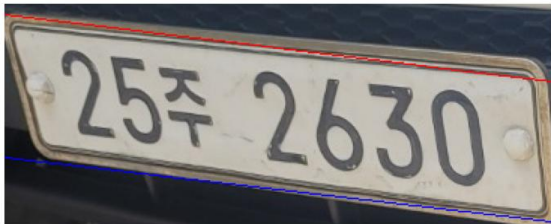
- Box_i : OCR이 인식한 글자 영역의 네 꼭짓점 좌표입니다.
- TL : 좌상단(Top-Left)
- TR : 우상단(Top-Right)
- BR : 우하단(Bottom-Right)
- BL : 좌하단(Bottom-Left)

$$Box_i = \{TL(x_1, y_1), TR(x_2, y_2), BR(x_3, y_3), BL(x_4, y_4)\}$$

Top / Bottom Lines



Top / Bottom Lines



빨간색- Top Line
파란색- Bottom Line

2. Top Line (상단 경계선) 구하기

1. 높이 저장 (t_i): 각 글자 상자의 상단 두 점(y_{TL}, y_{TR}) 중 더 높이에 있는(y 좌표가 작은) 값을 대표 높이 t_i 로 정합니다.

$$t_i = \min(y_{TL}, y_{TR})$$

2. 최상단 선택: 모든 상자 중 가장 위에 있는 상단의 선을 선택합니다.

$$\text{Selected Top Line} = \arg \min_{(Line, t) \in top_lines} (t)$$

3. 최종 확정: 선택한 선을 위로 10만큼 이동시켜 최종 Top Line으로 지정합니다.

3. Bottom Line (하단 경계선) 구하기

1. 높이 저장 (b_i): 각 글자 상자의 하단 두 점(y_{BL}, y_{BR}) 중 더 아래에 있는(y 좌표가 큰) 값을 대표 높이 b_i 로 정합니다.

$$b_i = \max(y_{BL}, y_{BR})$$

2. 최하단 선택: 모든 상자 중 가장 아래에 있는 하단의 선을 선택합니다.

$$\text{Selected Bottom Line} = \arg \max_{(Line, b) \in bottom_lines} (b)$$

3. 최종 확정: 선택한 선을 아래로 10만큼 이동시켜 최종 Bottom Line으로 지정합니다.


```
car_plate_number = None
number_candidate = []
car_plate_crop_width = w /*검출된 차량 번호판 이미지 너비*/
```

```
// 번호판 글자 잘림 방지를 위해 미세하게 영역을 조절하며 14회 반복
FOR i FROM 0 TO 13:
```

```
    // 1. 좌우 경계선을 안쪽으로 미세하게 좁힘
```

```
    left_line = get_left_line(i)
    right_line = get_right_line(w - i)
```

```
    // 2. 상하좌우 직선의 교점을 구하기
```

```
    new_box = get_intersection_points(top, bottom, left_line, right_line)
```

```
    // 3. 투영 변환(Warp)을 통해 번호판 영역을 수평으로 정렬된 이미지로 추출
```

```
    warped_img = warp_perspective(img, new_box)
```

```
    // 4. OCR 인식
```

```
    ocr_result = ocr.predict(warped_img)
```

```
    // 5. OCR 결과 문자 박스 필터링 및 중심점 계산
```

```
        Union-Find 알고리즘을 통해 필터링 된 문자 박스를 그룹화 및 그룹 내 정렬 진행
```

```
    // 6. 2줄 번호판, 1줄 번호판 후처리 진행
```

```
    // 7. 결과 후보군에 추가
```

```
    number_candidate.append("//6 에서의 후처리 결과")
```

```
    // 8. 정규표현식을 이용한 최종 자동차 번호 검증
```

```
    IF len(groups) == 2 AND matches_pattern(merged_text, "숫자2자리 + 한글 + 숫자4자리"): /*2줄 번호판 일 때*/
```

```
        car_plate_number = merged_text
```

```
        BREAK // 올바른 번호를 찾았으므로 루프 즉시 종료
```

```
    ELSE IF len(groups) == 1 AND matches_pattern(merged_text, "숫자2~3자리 + 한글 + 숫자4자리"): /*1줄 번호판 일 때*/
```

```
        car_plate_number = merged_text
```

```
        BREAK // 올바른 번호를 찾았으므로 루프 즉시 종료
```

```
// 9. 자동차 번호판 확정 및 반환
```

```
RETURN car_plate_number
```

왼쪽의 pseudo code는 Page 13에서 Top Line과 Bottom Line을 찾는 다음 실행되는 알고리즘이며, Page 14~18에 서는 실행 과정을 설명합니다.

page 16 pseudo code
// 번호판 글자 잘림 방지를 위해 미세하게 영역을 조절하며 14회 반복



검출된 출입 자동차 번호판

반복 0~13 동안 page 16 pseudo code 의 // 1 , // 2 , // 3 , // 4 를 실행한 결과

w는 page11에서 추출된 번호판 사진의 너비 입니다.



Left_line=0,
right_line=w



Left_line=1,
right_line=w-1



Left_line=2,
right_line=w-2



Left_line=3,
right_line=w-3



Left_line=4,
right_line=w-4



Left_line=5,
right_line=w-5



Left_line=6,
right_line=w-6



Left_line=7,
right_line=w-7



Left_line=8,
right_line=w-8



Left_line=9,
right_line=w-9



Left_line=10,
right_line=w-10



Left_line =11,
right_line =w-11



Left_line =12,
right_line =w-12



Left_line =13,
right_line =w-13

page 16 pseudo code

// 1 , // 2

탐색 범위: $i \in \{0, 1, 2, \dots, 13\}$ (총 14단계 변형)

1) 좌측 수직 경계선

조건: $x_{min} = 0$ 이므로 수식은 다음과 같이 전개됩니다.

$$1 \cdot x + 0 \cdot y = 0 + i$$

$$\therefore x = i$$

w는 page11에서 추출된 번호판 사진의 너비 입니다.

2) 우측 수직 경계선

조건: $x_{max} = w$ (이미지의 전체 가로 너비) 이므로 수식은 다음과 같이 전개됩니다.

$$1 \cdot x + 0 \cdot y = w - i$$

$$\therefore x = w - i$$



Left_line=0,
right_line=w



Left_line=1,
right_line=w-1



Left_line=2,
right_line=w-2



Left_line=3,
right_line=w-3



Left_line=4,
right_line=w-4



Left_line=5,
right_line=w-5



Left_line=6,
right_line=w-6



Left_line=7,
right_line=w-7



Left_line=8,
right_line=w-8



Left_line=9,
right_line=w-9



Left_line=10,
right_line=w-10



Left_line =11,
right_line =w-11



Left_line =12,
right_line =w-12



Left_line =13,
right_line =w-13

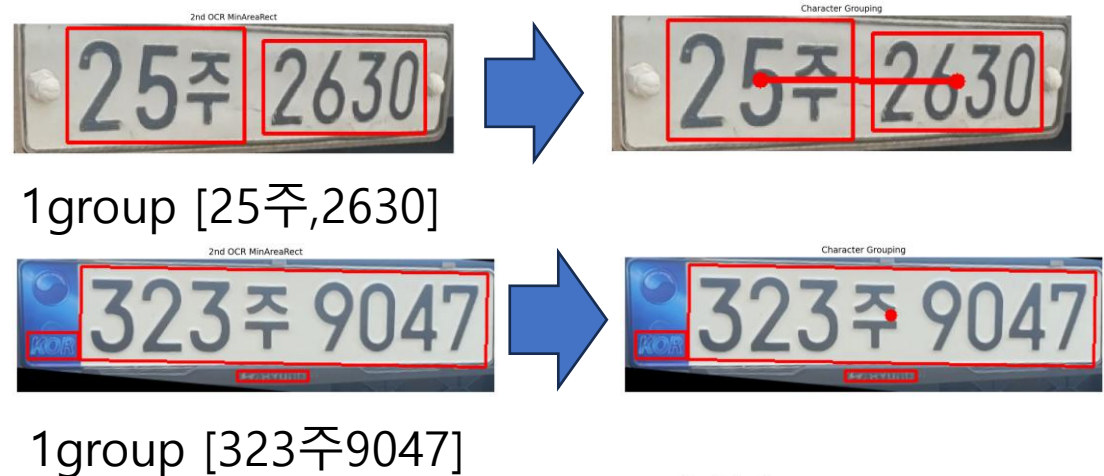
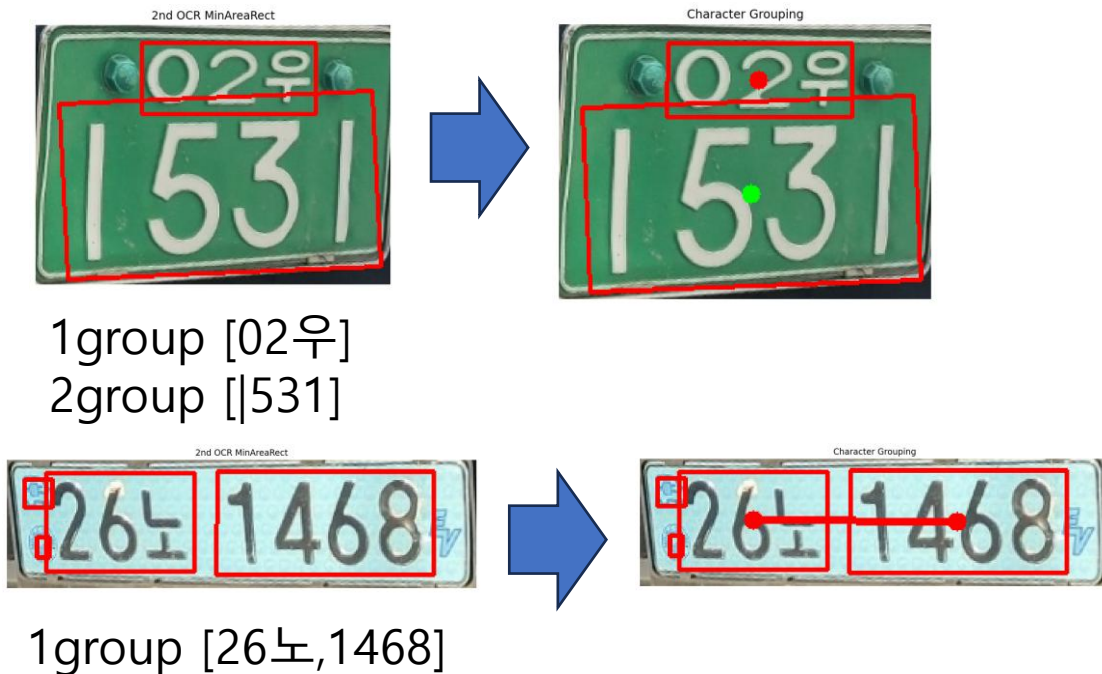
page 16 pseudo code // 3

"좌우 마진이 반영된 4개의 교점 좌표(TL, TR, BR, BL)를 추출하고
open cv 의 투영 변환(Perspective Transform)을 통해
왜곡된 번호판 이미지를 정면 보정(Warping)합니다."



page 16 pseudo code // 4 , // 5

1. Warping한 이미지를 **PaddleOCR**에 적용하여 문자 영역과 인식 결과를 추출하였습니다.
2. 검출된 문자 영역을 **open cv의 MinAreaRect**를 이용하여 최소 외접 직사각형 형태로 변환하였습니다.
3. 문자 영역(최소 외접 직사각형 형태로 변환)의 너비가 Warping한 이미지 너비의 15% 이하이거나, 높이가 Warping한 이미지 높이의 15% 이하인 경우 중심점을 부여하지 않았습니다.
4. 중심점이 부여된 문자 영역에 대하여 중심 좌표를 계산하였습니다.
5. 각 문자 중심점 간의 방향 벡터를 생성하고, 기준 벡터 (1,0)과의 코사인 유사도를 계산하였습니다.
6. 코사인 유사도가 0.99 이상인 경우 동일한 행(Row)에 존재하는 문자로 판단하였습니다.
7. 동일한 행으로 판단된 문자들은 **Union-Find 알고리즘**을 이용하여 하나의 그룹으로 병합하였습니다.
8. 그룹 내부의 문자들은 중심점의 x좌표를 기준으로 좌측에서 우측 순으로 정렬하였습니다.
9. 생성된 그룹들은 중심점의 y좌표를 기준으로 상단에서 하단 순으로 정렬하였습니다.



2줄 번호판 (group 2)

1 그룹, 2 그룹 모두 적용

 | 로 오인식 나는 경우가 있어 | 를 1 로 변경하였습니다.

 O, o, o 으로 오인식 나는 경우가 있어 O, o, o 를 0으로 변경하였습니다.

숫자와 한글 이외의 문자 및 공백을 제거합니다

1 그룹 부분에만 적용

지역명: 서울,경기,인천,강원,대전,충북,충남, 세종,광주,전북,전남,부산,대구,울산, 경북,경남,제주

지역명과 그 앞의 문자까지 제거

남는 문자의 길이가 3이면 맨 마지막 문자 제거

정규표현식을 사용하여 숫자들 + 한글 1개 + 숫자들 형태를 탐색하였다.

나사를 숫자로 오인식 된 부분 한글1개 뒤에 있는 숫자를 제거하였다.

2줄 번호판 첫 번째 줄은 일반적으로 숫자 2자리 + 한글 1자리 형태를 가진다.

OCR이 나사, 볼트, 그림자 등을 숫자로 오인식 하여 앞자리가 추가되는 경우가 발생한다. 한글 앞 숫자가 3자리로 검출되면 첫 번째 숫자를 제거하여 번호판 형식에 맞게 정제하였다.



1줄 번호판 (group 1)

1 그룹 부분에만 적용

그룹에 있는 문자들을 합칩니다.

숫자와 한글 이외의 문자 및 공백을 제거합니다.

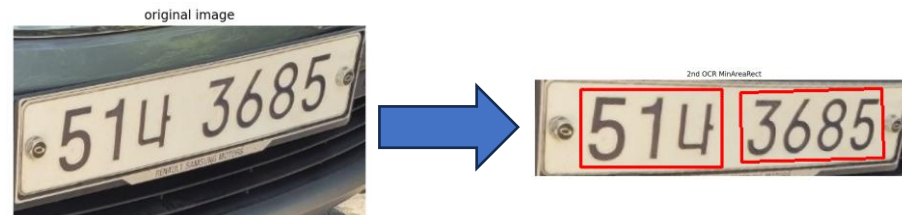
맨 앞에 있는 숫자가 아닌 문자들을 모두 제거한다.

경기12사1234 -> 12사1234

번호판에 있는 나 글자가 숫자 4로 오인식 되는 경우가 있어서

7자리->3번째 자리에 4이면 4를 나로 변경

5143658 -> 51나3658

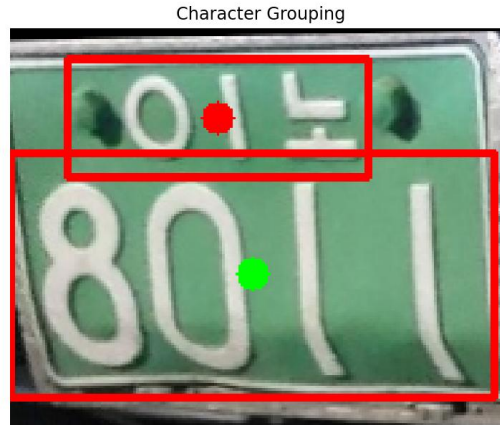


8자리-> 4번째 자리에 4이면 4를 나로 변경

12241234 -> 122나1234

실패 사례 ,결론 및 향후계획

실패 사례



```
texts ['01노', '8011']
```

page 16 pseudo code 의 // 1 , // 2 , // 3 , // 4 , //5 를 실행한 결과

```
number_candidate ['0|8011', '0|8011', '0|8011', '0|8011', '0|8011', '0|8011', '0|8011',  
'0|8011', '0|8011', '0|8011', '0|8011', '0|8011', '0|8011']  
car_number: None
```

page 16 pseudo code 의 // 6 , // 7 , // 8 , // 9를 실행한 결과

실제 주차장 출입 환경을 가정한 야간 이미지에서 차량과 번호판 영역은 정상적으로 탐지되었지만, 사람의 눈으로 보았을 때는 01노8011 이지만 OCR 단계에서 01 을 '이'로 오 인식되어 최종 차량 번호를 생성하지 못하였다. 따라서, 번호판 형식 검증을 통과하지 못해 car_number는 None으로 처리되는 현상이 발생 하였다.

결론

이번 프로젝트에서는 차량 영역을 먼저 추출한 뒤 번호판을 탐지하는 2단계 탐지 방식을 적용하여, 전체 이미지에서 발생하던 차량 로고·표지판 등의 오탐을 줄이고 번호판 탐지의 안정성을 향상시켰다. 다만, 본 프로젝트는 맑은 환경 중심 및 야간 환경에서 선명하게 보이는 번호판 데이터로 실험을 진행하였으며, 실제 주차장에서 흔하게 발생할 수 있는 흐릿한 영상, 우천·안개, 강한 조명 반사 등의 환경은 충분히 고려하지 못한 한계가 있다.

향후 개선 방향

자동차 번호판 인식을 위해 별도의 이미지 전처리(Image Preprocessing)를 적용하지 않고 원본 영상을 기반으로 번호판 탐지 및 OCR을 수행하였다.

향후에는 저조도, 우천, 흐림(Blur) 환경에서도 문자 대비를 향상시킬 수 있는 이미지 전처리 기법을 연구하여, 실제 주차장 CCTV 환경에서의 번호판 인식 성능을 개선하고자 한다.

학습 데이터가 번호판 중심 이미지에 편중되어 있음에도 실제 주차장 출입 환경에서 차량 전면과 다양한 배경이 포함된 장면에서도 번호판을 오탐 없이 안정적으로 탐지할 수 있도록 일반화 성능을 향상시키는 연구를 진행하고자 한다.

구형 자동차 번호판에서 나사(Screw)가 문자 또는 숫자로 오인식 되고, 01노8011의 앞자리 01이 이로 인식되는 paddle OCR 오류를 확인하였다.

또한, 신형 번호판에서는 한글 문자 나가 숫자 4로 오인식 되는 사례도 확인되어 자동차 번호 규칙 기반으로 4를 '나'로 치환하는 후처리 규칙을 적용하였다.

따라서 위와 같은 발생하는 OCR 오류를 분석하고 구형 및 신형 번호판의 특성을 고려한 OCR 모델 최적화를 수행할 예정이다. 최종적으로 번호판 문자 인식의 정확도를 향상시키고자 한다.

주차장 출입 차량의 **전면 번호판을 대상으로 번호판 인식 시스템을 구현하였으며, 후면 번호판에 대한 인식은 고려하지 않았다.** 향후에는 전·후면 번호판의 형태적 특성을 고려한 인식 기술을 적용하여 차량 출입 인식 시스템의 활용 범위를 확대할 예정이다.